# 分岐後挙動の比較に基づく Windowsマルウェアの VM・Sandbox回避機能調査

岡村 晃誠[1,2,a)]　梅村 勇貴[1]　古川 秀之[1]　毛利 公一[1,2]　久保 正樹[1]　笠間 貴弘[1]

**概要**：マルウェアは，VM や Sandbox の構成・運用に伴う環境上の痕跡を検査し，解析環境と判断すると悪性挙動を抑止して動的解析を回避することがある．VM が実運用環境にも普及し，Sandbox の構成・運用も変化する中で，解析環境の判定に利用される痕跡や回避手法の構成も変化している可能性がある．そこで本研究では，マルウェアの VM・Sandbox 回避機能を抽出し，異なる時期における回避機能の利用状況と回避手法を比較する．環境検査処理とその結果に依存する条件分岐を手掛かりとする既存手法では，解析回避を目的とした分岐と，通常の互換性確認やエラー処理を区別できないことがある．提案手法では，環境検査に依存する分岐後の各経路で実行される処理を LLM によって比較・解釈し，解析環境と判定された経路で悪性挙動が抑止されるかに基づいて VM・Sandbox 回避機能を判定する．2025–2026 年の判定可能な 659 検体に適用した結果，217 検体で環境検査結果に依存する条件分岐が確認され，分岐後挙動まで考慮することで 81 検体が回避機能候補として抽出された．人手解析ではこのうち 64 検体で VM・Sandbox 回避機能が確認され，分岐後挙動に基づく判定の適合率は 79.0%であった．判定可能検体における回避機能の確認割合は 2020–2021 年で 12.25%，2025–2026 年で 9.71%であり，統計的に有意な差は確認されなかった．

**キーワード**：マルウェア解析，VM 回避，Sandbox 回避，LLM，静的解析

# Detecting and Analyzing VM and Sandbox Evasion in Windows Malware through Post-Branch Behavior Comparison

Kosei Okamura[1,2,a)]　Yuki Umemura[1]　Hideyuki Furukawa[1]　Koichi Mouri[1,2]　Masaki Kubo[1]
Takahiro Kasama[1]

***Abstract:*** Malware may evade dynamic analysis by inspecting environmental artifacts associated with virtual machines (VMs) and sandboxes and suppressing malicious behavior when an analysis environment is detected. As VMs have become common in production and sandbox configurations and operations have evolved, the artifacts and techniques used to identify analysis environments may also have changed. This study investigates VM and sandbox evasion capabilities in Windows malware and compares their prevalence and techniques across different periods. Existing methods based on environmental checks and result-dependent branches may fail to distinguish evasion from ordinary processing. We therefore use a large language model (LLM) to compare and interpret post-branch behavior and identify evasion when malicious behavior is suppressed on the analysis-environment-detected path. Applying the proposed method to 659 Windows malware samples from 2025–2026 identified 217 samples with environment-dependent branches, of which 81 were identified as suppressing malicious behavior based on post-branch behavior. Manual analysis confirmed VM and sandbox evasion in 64 of these 81 samples. Across the two periods, evasion was confirmed in 12.25% of samples from 2020–2021 and 9.71% from 2025–2026, with no significant difference. Timing-based techniques were more common earlier, whereas artifacts related to analysis tools were more common later.

***Keywords:*** Malware Analysis, VM evasion, Sandbox evasion, Large Language Models, Static Analysis

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

## 1. はじめに

マルウェアの機能や攻撃過程を把握する手法の一つとして，検体を VM や Sandbox などの解析環境上で実行し，その挙動を観測する動的解析が広く用いられている [1]．一方，VM や Sandbox を用いた解析環境には，仮想化されたハードウェアやソフトウェア構成，解析用コンポーネントなど，実行環境を識別するための特徴が存在する．マルウェアはこれらの特徴を検査し，解析環境であると判断した場合に処理の終了や悪性挙動の抑止などを行うことで，本来の悪性挙動を発現させず，動的解析を回避することがある [2], [3]．本研究では，VM の利用や Sandbox の構成・運用に伴う環境上の特徴を用いて解析環境を識別し，その判断に基づいて悪性挙動を抑止する機能を VM・Sandbox 回避機能と呼ぶ．

VM はマルウェア解析だけでなく，サーバ仮想化やクラウドコンピューティングを支える基盤として，実運用環境にも広く利用されている [4], [5]．このため，VM の存在のみを理由に実行を抑止することは，攻撃対象となり得る実運用環境まで除外し，マルウェアにとって攻撃機会の損失につながる可能性がある．また，解析環境には VM の有無だけでなく，解析ツール，セキュリティ製品，システム資源，ユーザ構成など，解析環境を識別するための多様な痕跡が存在する．したがって，実運用環境や解析環境の変化に伴い，マルウェアが解析環境の判定に利用する特徴や VM・Sandbox 回避手法の構成が，過去の調査時点から変化している可能性がある．既存の縦断調査においても，解析回避手法の利用傾向には時期による差異が報告されている [6]．異なる時期に観測されたマルウェアを同一の基準で比較し，VM・Sandbox 回避機能の確認割合や，解析環境の判定に利用される特徴の差異を把握することは，解析環境の設計や回避対策において考慮すべき痕跡を整理するうえで有用である．

このような年代比較を行うためには，各検体が VM・Sandbox 回避機能を有するか否かを，一貫した基準で判定する必要がある．既存研究では，解析環境に関連する特徴の検査や，その結果に依存する実行経路の変化を手掛かりとして，解析回避機能の利用状況が調査されている [6], [7], [8]．しかし，環境情報を取得し，その結果に応じて処理を分岐しているだけでは，VM・Sandbox 回避機能を有すると判断するには不十分である．例えば，実行環境に応じて通常処理を切り替える場合や，環境情報の取得に失敗した際のエラー処理でも，同様の条件分岐が生じ得る．このため，環境検査処理やその結果に依存する条件分岐のみを判定基準とした場合，VM・Sandbox 回避を目的としない処理が回避機能として含まれ，年代間比較の妥当性を損なう可能性がある．VM・Sandbox 回避機能を一貫した基準で比較するためには，解析環境の判定に利用される特徴を確認するだけでなく，解析環境と判定された経路において悪性挙動が抑止されるかを確認する必要がある．

そこで本研究では，環境検査結果に依存する条件分岐を追跡し，各分岐先の処理内容から悪性挙動の抑止を判定することで，VM・Sandbox 回避機能を抽出する手法を提案する．分岐後の処理は検体ごとに多様であり，固定的なシグネチャのみからその意味を一律に判定することは困難であるため，処理内容の解釈に LLM を用いる．提案手法を異なる時期に観測された Windows マルウェアに同一の基準で適用し，抽出結果を人手で確認するとともに，VM・Sandbox 回避機能の確認割合と回避手法を調査する．さらに，年代間で得られた結果を比較し，確認割合および解析環境の判定に利用される特徴の構成にどのような差異があるかを分析する．

以上を踏まえ，本研究では 2 つの Research Question（RQ）を設定する．RQ1: 環境検査結果に依存する分岐が確認された検体に対し，分岐後挙動に基づく判定によって VM・Sandbox 回避機能をどの程度識別できるか．RQ2: 同一の解析・確認手順を適用したとき，2020–2021 年と 2025–2026 年の検体間で，VM・Sandbox 回避機能が確認される割合およびその回避手法にどのような差異があるか．

本研究の貢献は以下の三点である．第一に，環境検査処理やその結果に依存する条件分岐だけでなく，分岐後の処理内容を考慮して VM・Sandbox 回避機能を判定する手法を提案する．第二に，分岐後挙動を考慮することで，環境検査結果に依存する分岐のみを基準とする場合に含まれる，回避を目的としない処理を候補から除外できることを示した．第三に，2020–2021 年と 2025–2026 年では VM・Sandbox 回避機能の確認割合に有意な差はない一方，前者では時間計測やエミュレーション特有の挙動，後者では解析ツール・セキュリティ製品や VM・Sandbox 固有の痕跡を利用する手法が多く確認されるなど，回避手法の構成に異なる傾向が観測された．

## 2. 背景および関連研究

### 2.1 VM・Sandbox 回避機能

マルウェアの動的解析に用いられる VM や Sandbox には，その構築や運用に伴う環境上の特徴が存在する．VM では仮想化されたハードウェアやソフトウェア構成が実行環境を推定する手掛かりとなる．また，Sandbox では，解析用のプロセスやモジュール，セキュリティ製品，システム資源，ホストやユーザの構成，ユーザ操作などに加え，エミュレーションや API 実装に起因する挙動差，解

[1] 国立研究開発法人情報通信研究機構
National Institute of Information and Communications Technology (NICT)
[2] 立命館大学
Ritsumeikan University
[a)] kokamura@nict.go.jp

© 2026 Information Processing Society of Japan

析・監視機構などが実行環境を識別する手掛かりとなり得る [6], [8]．マルウェアはこれらの特徴を検査し，解析環境であると判断した場合に悪性挙動を抑止することで，動的解析を回避することがある．

VM と Sandbox では解析環境の判定に利用される特徴が重複する場合があり，VM 上に構築された Sandbox では両者に由来する特徴が同時に存在する．また，VM が実運用環境にも普及していることから，VM 固有の痕跡だけでなく，Sandbox の構成や運用に伴う複数の特徴を組み合わせて解析環境を判定する可能性も考えられる．このため，本研究では VM と Sandbox を厳密に分離せず，VM の利用または Sandbox の構成・運用に伴う特徴を解析環境の識別に利用し，その判断に基づいて悪性挙動を抑止する機能を VM・Sandbox 回避機能として扱う．

ここで，解析ツールやセキュリティ製品，ホスト・ユーザ情報等を検査する処理であっても，その存在のみをもって VM・Sandbox 回避機能とは判定しない．これらの特徴が解析環境を識別するために利用され，その結果に基づいて悪性挙動が抑止されることを確認できる場合を本研究の対象とする．なお，デバッガの存在や接続状態を検査する Anti-Debugging は本研究の対象外とする．

### 2.2 既存研究と本研究の位置付け

Windows マルウェアによる解析回避については，多数の検体を対象とした調査が行われている．本節では，環境検査から後続処理までのどの範囲を回避機能の判定に用いているかに着目し，既存研究と本研究の位置付けを整理する．

Galloro らは，92 種類の既知の解析回避手法を対象として，2010 年から 2019 年までの 45,375 検体を解析した [6]．少なくとも一つの対象手法が確認された検体の割合は 2010 年の 67.8%から 2019 年の約 80%へ増加し，利用される解析回避手法の構成にも年代による差異が報告されている．

Maffia らは，Pepper を用いて 7 カテゴリ 53 種類の解析回避手法を対象に 183,340 検体を解析し，2016 年から 2020 年における利用状況を調査した [7]．解析回避手法全体の利用が増加する一方で VM Checks は減少傾向にあり，その要因として，サーバ仮想化の普及によって仮想環境を一律に感染対象から除外することが不利益となる可能性を考察している．

Kim らは，環境検査に関連する API の戻り値や出力値を追跡し，その値が条件分岐に影響することを確認する EvDetector を提案した [8]．2017 年から 2020 年までの 763,985 検体への適用では，対象とした環境検査処理の結果が条件分岐に利用された検体の割合を平均 16.21%と報告している．EvDetector は，環境検査処理の存在に加えて，その結果がプログラムの実行制御に利用されることまで確認する．

表 1 に，これらの既存研究と本研究における回避機能の判定範囲を示す．Galloro らおよび Maffia らは環境検査処理を，Kim らはさらに検査結果に依存する条件分岐までを判定に用いており，いずれの研究も，分岐後に実行される処理の内容に基づいて回避機能を判定するものではない．

**表 1** 既存研究と本研究における回避機能の判定範囲

| 研究 | 環境検査処理 | 結果依存分岐 | 分岐後挙動に基づく回避判定 |
|---|---|---|---|
| Galloro ら | ○ | – | – |
| Maffia ら | ○ | – | – |
| Kim ら | ○ | ○ | – |
| 本研究 | ○ | ○ | ○ |

これに対し本研究では，環境検査結果に依存する条件分岐を特定した後，各分岐先の処理内容を比較し，解析環境と判定された経路における悪性挙動の抑止に基づいて VM・Sandbox 回避機能を判定する．分岐後の処理は検体ごとに多様であるため，その処理内容の解釈には LLM を用いる．さらに，本手法を異なる時期に観測された Windows マルウェアに同一の解析・確認手順で適用し，VM・Sandbox 回避機能の確認割合および解析環境の判定に利用される特徴の構成を比較する．

## 3. VM・Sandbox 回避機能抽出手法

### 3.1 概要

本研究では，Windows マルウェアに含まれる VM・Sandbox 回避機能を抽出するため，既知の VM・Sandbox 関連処理に対応するシグネチャから解析候補を抽出し，環境検査結果に依存する条件分岐と，その分岐後に実行される処理を段階的に解析する．環境検査結果に依存する条件分岐の存在のみを判定基準とせず，その分岐後の両経路を追跡・比較し，解析環境を検出した経路において悪性挙動が抑止されることまでを確認して VM・Sandbox 回避機能を判定する．これにより，通常の環境依存分岐と，解析環境の検出によって悪性挙動を抑止する回避処理を区別する．

環境検査結果の利用関係や分岐後の処理は検体ごとに異なるため，本研究では，LLM に MCP を介して Ghidra を操作させ，データフローおよび分岐後の実行経路を追跡する．近年では，LLM エージェントが MCP を介してバイナリ解析ツールを操作し，マルウェアの挙動を抽出する手法も提案されている [9]．本研究では各 Level の判定条件をあらかじめ定め，LLM はその判定に必要なコードおよび参照関係の取得，データフロー・制御フローの追跡，処理内容の解釈に用いる．

図 1 に，本研究における解析システムの構成を示す．

解析は 3 段階で行い，各候補が満たした判定条件に基づいて Level 1 から Level 3 を付与する．シグネチャとの一致が確認されなかった検体は Level 0 とする．各 Level の定義を表 2 に示す．

© 2026 Information Processing Society of Japan

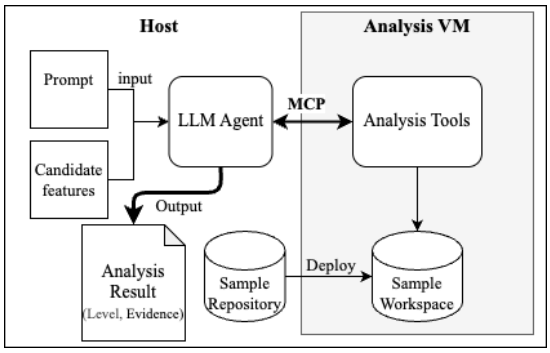



**図 1** VM・Sandbox 回避機能解析システムの構成

**表 2** 解析 Level の定義

| Level | 判定内容 |
|---|---|
| 0 | シグネチャとの一致なし |
| 1 | VM・Sandbox 関連処理の候補あり |
| 2 | 検査結果に依存する条件分岐あり |
| 3 | 分岐後に悪性挙動の抑止あり |

Level 1 から Level 3 までは候補ごとに判定し，1 検体から複数の候補が得られた場合には，各候補が到達した最大の Level を検体の Level とする．

### 3.2 VM・Sandbox 関連処理の候補抽出

VM・Sandbox 関連処理の候補抽出には，Unprotect Project [10] で整理されている解析回避手法を利用する．解析条件を固定するため，2026 年 4 月 1 日時点の手法情報を取得し，本研究の対象に該当する手法に関連する API，および特定の命令パターンを抽出した．本研究では，これらを Level 1 の候補を抽出するためのシグネチャと呼ぶ．使用したシグネチャと対応する Unprotect 手法を付録 A.3 の表 A·1 に示す．

解析対象からシグネチャとの一致を探索し，一致した処理を Level 1 の候補とする．API については静的な import だけでなく，GetProcAddress による動的解決や API hashing によって解決される API も対象とする．関連する文字列は単独ではシグネチャとせず，一致した手法における検査対象を特定するための補助情報として用いる．例えば，GetTickCount や FindWindowA の API 呼出し，RDTSC や CPUID の命令をシグネチャとして用いる．

### 3.3 検査結果に依存する分岐の解析

Level 1 の各候補について，候補となった API または命令の使用箇所を起点として解析する．まず，各手法において環境検査の結果を表す値を特定し，その値が後続の処理でどのように利用されるかを追跡する．具体的には，API の戻り値や出力値，命令によって取得された値，またはそれらから算出された値のデータフローを追跡し，これらの値が条件分岐の判定に利用されているかを確認する．単に環境検査処理と条件分岐が近接していることは判定根拠とせず，検査結果を表す値またはその派生値と条件分岐とのデータ依存関係を確認できた候補を Level 2 とする．

### 3.4 分岐後挙動に基づく回避機能判定

Level 2 で特定した条件分岐について，解析環境を検出した場合に対応する経路と，検出しなかった場合に対応する経路をそれぞれ追跡し，分岐後に実行される処理を比較する．各経路は，両経路が再合流する，処理が終了する，または両経路の挙動差を判断できる地点まで追跡する．

分岐後の処理については，呼出し先関数，API，参照関係および制御フローを追跡し，両経路の処理内容の差と，検出しなかった経路で実行される悪性挙動を特定する．解析環境を検出した経路において，この悪性挙動が実行されない場合を悪性挙動の抑止とみなし，当該候補を Level 3 とする．具体的には，悪性処理の実行前の終了，ペイロード展開・復号・実行の省略，悪性挙動の実行を妨げる長時間の待機や無限ループを回避挙動として扱う．ただし，通常のエラー処理，互換性確認，実行環境に応じた通常処理の選択，および両経路が再合流して同一の悪性挙動を実行する場合は Level 3 とは判定しない．

Level 3 の判定では，環境検査結果に依存する条件分岐，解析環境を検出した側の経路，検出しなかった側の経路についてそれぞれ根拠となるコードを取得し，両経路の処理内容の差と抑止される悪性挙動を特定する．検出しなかった経路で実行される処理が悪性挙動であること，およびその悪性挙動が解析環境を検出した経路で抑止されることを十分な根拠から確認できない場合には判定不能とする．

## 4. 評価

本章では，第 1 章で示した RQ1 および RQ2 を評価する．

### 4.1 評価データセット

評価には，VirusTotal [11] への初回投稿日時が 2020 年 4 月 1 日から 2021 年 3 月 31 日までの検体と，2025 年 4 月 1 日から 2026 年 3 月 31 日までの検体を用いる．両期間とも 1 年間を対象とし，同一の検索条件を用いて Windows PE 実行ファイルを収集した．

収集条件として，VirusTotal 上で Windows PE 実行ファイルとして識別され，30 以上のアンチウイルス製品からマルウェアとして検知された検体を対象とした．各期間について条件を満たす 10,000 検体の集合を作成し，各検体集合に固定した乱数シードに基づく無作為な順位を付与した上で，その順位順に解析した．なお，解析検体数は，利用可能な解析期間および計算資源上の制約により，2 つの検体集合で異なる．解析の結果，検査結果の意味，分岐先との対応，到達性，または後続経路を十分に確認できなかった検体は判定不能とした．2020–2021 年では 429 検体中 400 検体，2025–2026 年では 701 検体中 659 検体について判定

© 2026 Information Processing Society of Japan

表 3　評価に用いる検体集合

| 対象期間 | 抽出数 | 判定可能 | 判定不能 |
|---|---|---|---|
| 2020/04/01–2021/03/31 | 429 | 400 | 29 |
| 2025/04/01–2026/03/31 | 701 | 659 | 42 |

可能であり，それぞれ 29 検体および 42 検体を判定不能として以降の評価から除外した．表 3 に，各期間の評価対象検体を示す．

両期間の検体に対して，同一のシグネチャ集合および判定基準を用いて提案手法を適用した．バイナリ解析には mecha_ghidra を用い，分岐後挙動の解析には OpenAI GPT-5.5 を使用した．reasoning effort は high に設定し，両期間について同一のモデルおよび解析設定を用いた．

### 4.2　RQ1: 分岐後挙動に基づく判定の妥当性

#### 4.2.1　評価方法

RQ1 では，2025–2026 年の判定可能な 659 検体のうち，環境検査結果に依存する条件分岐が確認された Level 2 以上の検体を対象として，分岐後挙動に基づく Level 3 判定の妥当性を評価する．

Level 2 以上のすべての検体について人手による解析を行った．人手解析では，提案手法が出力したシグネチャの使用箇所および関連するコード位置を解析の起点として用いる一方，Level 判定結果には依存せず，Ghidra 上で環境検査処理，結果依存分岐および分岐後処理を追跡した．Pack された検体については，解析パイプラインで取得した unpack 後のバイナリを用いた．その上で，3.4 節と同一の判定基準に基づき，解析環境を検出した側において悪性挙動の全部または一部が実質的に抑止されることを確認できた検体について，VM・Sandbox 回避機能を有すると判定した．

人手による解析結果を参照判定とし，Level 3 と判定され，人手でも VM・Sandbox 回避機能を確認した検体を真陽性（TP），Level 3 と判定されたものの人手では回避機能を確認できなかった検体を偽陽性（FP）とする．また，Level 2 に分類された検体のうち，人手では VM・Sandbox 回避機能を確認した検体を偽陰性（FN），人手でも回避機能を確認できなかった検体を真陰性（TN）とする．

これらの結果から Level 3 判定の適合率，再現率および F1 値を算出する．ただし，Level 0 および Level 1 に分類された検体については網羅的な人手解析を行っていないため，本評価で算出する再現率は Level 2 以上の検体に対する Level 3 判定の再現率であり，候補抽出を含む解析パイプライン全体の再現率ではない．

#### 4.2.2　結果

2025–2026 年の 659 検体のうち，217 検体で環境検査結果に依存する条件分岐が確認され，Level 2 以上と判定された．このうち，分岐後の両経路を解析した結果，81 検体が Level 3，136 検体が Level 2 と判定された．

表 4　Level 判定結果と人手確認結果の比較

| LLM 判定 | 回避機能あり | 回避機能なし | 確認割合 |
|---|---|---|---|
| Level 2 | 0 | 136 | 0.0% |
| Level 3 | 64 | 17 | 79.0% |
| Level 2 以上 | 64 | 153 | 29.5% |

表 4 に，Level 3 判定と人手確認結果の対応を示す．

Level 3 の 81 検体中 64 検体で VM・Sandbox 回避機能を確認した一方，Level 2 に留まった 136 検体では回避機能は確認されなかった．この結果，Level 3 判定の適合率は 79.0%（64/81），再現率は 100.0%（64/64），F1 値は 88.3% であった．

環境検査結果に依存する条件分岐までを基準として Level 2 以上の 217 検体を回避機能候補とした場合，人手によって実際に VM・Sandbox 回避機能が確認された割合は 29.5%（64/217）であった．これに対し，分岐後挙動まで考慮した Level 3 判定では候補が 81 検体まで絞り込まれ，そのうち 79.0%で回避機能が確認された．

偽陽性となった 17 検体には，VM・Sandbox 回避にも利用される汎用的な API や処理構造を，解析回避を目的とした処理として解釈した例が含まれていた．代表例として，GetTickCount や QueryPerformanceCounter を用いる MSVC の__security_init_cookie()，解析ツール以外を探索する FindWindow 系 API，ユーザ名に応じて権限確認処理を変更する処理，デバイスの有無に応じて処理を終了する処理が確認された．

### 4.3　RQ2: 回避機能の確認割合と手法の年代比較

#### 4.3.1　評価方法

RQ2 では，2020 年 4 月 1 日から 2021 年 3 月 31 日までの検体と，2025 年 4 月 1 日から 2026 年 3 月 31 日までの検体について，同一の解析・確認手順の下で VM・Sandbox 回避機能の確認割合および回避手法を比較する．

まず，両期間の判定可能検体に提案手法を適用し，Level 0 から Level 3 までの検体数および割合を比較する．次に，Level 2 以上と判定された検体を人手で解析し，判定可能検体のうち VM・Sandbox 回避機能が確認された検体の割合を算出する．各期間の確認割合について 95%信頼区間を算出し，両期間の差について両側 Fisher の正確確率検定を有意水準 5%で行う．

さらに，人手によって VM・Sandbox 回避機能を確認した検体について，解析環境の判定に利用される特徴または回避挙動の性質に基づいて回避手法を 8 カテゴリに分類する．分類には，時間・エミュレーション挙動，VM・Sandbox 固有の痕跡，解析ツール・セキュリティ製品，システム資源，ホスト・ユーザ環境，ユーザ操作，API の挙動差，実行時監視機構を用いる．分類は API や命令そのも

© 2026 Information Processing Society of Japan

表 5　時期別の Level 判定結果

| 期間 | Level 0 | Level 1 | Level 2 | Level 3 |
|---|---|---|---|---|
| 2020–21 | 76 (19.0%) | 164 (41.0%) | 95 (23.8%) | 65 (16.3%) |
| 2025–26 | 257 (39.0%) | 185 (28.1%) | 136 (20.6%) | 81 (12.3%) |

表 6　時期別の VM・Sandbox 回避機能の確認結果

| 対象期間 | 判定可能数 | 確認数 | 確認割合 |
|---|---|---|---|
| 2020/04–2021/03 | 400 | 49 | 12.25% |
| 2025/04–2026/03 | 659 | 64 | 9.71% |

のではなく検体内での利用目的に基づいて行い，1 検体から複数の手法が確認された場合には重複して集計する．

#### 4.3.2　VM・Sandbox 回避機能の確認割合

表 5 に，各期間における Level 別の解析結果を示す．

2020–2021 年の 400 検体では Level 0，Level 1，Level 2，Level 3 がそれぞれ 76 検体，164 検体，95 検体，65 検体であった．2025–2026 年の 659 検体では，それぞれ 257 検体，185 検体，136 検体，81 検体であった．

Level 2 以上と判定された検体を人手で確認した結果，VM・Sandbox 回避機能を確認した検体は 2020–2021 年で 49 検体，2025–2026 年で 64 検体であった．表 6 に，判定可能検体における VM・Sandbox 回避機能の確認結果を示す．

VM・Sandbox 回避機能の確認割合は，2020–2021 年で 12.25%（95%信頼区間：9.39–15.83%），2025–2026 年で 9.71%（95%信頼区間：7.68–12.21%）であった．両期間の差について Fisher の正確確率検定を行った結果，有意差は確認されなかった（$p = 0.218$）．

#### 4.3.3　回避手法の構成

表 7 に，各期間で人手によって確認された VM・Sandbox 回避手法と，各カテゴリで確認された代表的な API，命令，環境指標を示す．1 検体で複数の手法が確認された場合には，各カテゴリへ重複して集計した．

2020–2021 年では，VM・Sandbox 回避機能を確認した 49 検体のうち，Timing/emulation behavior が 24 検体（49.0%）で最も多く，Host/user environment が 11 検体（22.4%），Analysis tools/security products が 9 検体（18.4%）であった．

2025–2026 年では，VM・Sandbox 回避機能を確認した 64 検体のうち，Analysis tools/security products が 26 検体（40.6%）で最も多く，VM/Sandbox artifacts が 20 検体（31.3%），System resources および Host/user environment がそれぞれ 12 検体（18.8%）であった．

このように，2020–2021 年では Timing/emulation behavior が多く確認されたのに対し，2025–2026 年では Analysis tools/security products および VM/Sandbox artifacts が多く確認され，両期間で確認された回避手法の構成に異なる傾向がみられた．

## 5.　考察

### 5.1　分岐後挙動に基づく判定の有効性

2025–2026 年の評価では，環境検査結果に依存する分岐までを基準とした Level 2 以上の検体には，VM・Sandbox 回避を目的としない処理が多く含まれていた．分岐後挙動まで考慮した Level 3 判定では，人手によって VM・Sandbox 回避機能が確認された割合が 29.5%から 79.0%に向上しており，環境検査結果に依存する分岐の存在だけでなく，その後に悪性挙動が抑止されるかを確認することが，回避機能の識別に有効であった．

一方，Level 3 判定にも偽陽性があり，VM・Sandbox 回避にも利用される汎用的な API や処理構造を，解析回避を目的とした処理として解釈した例が確認された．例えば，MSVC の `__security_init_cookie()` では，`GetTickCount` や `QueryPerformanceCounter` 等を用いる処理を環境検査として誤って解釈していた．また，解析ツール以外を対象としたウィンドウ探索や，ユーザ名，デバイスの有無に応じた処理の変更を回避処理と判定した例もあった．このような偽陽性に対しては，コンパイラやランタイムに由来する既知の処理をあらかじめ識別するとともに，分岐条件となる検査対象が VM・Sandbox 等の解析環境と関連するかを確認することで，判定精度を改善できる可能性がある．

Galloro らは，環境検査処理を含む既知の解析回避手法を基準として，対象手法が確認された検体の割合を 2010 年の 67.8%から 2019 年の約 80%と報告している [6]．本研究でこれに対応する Level 1 以上の割合は，2020–2021 年で 81.0%，2025–2026 年で 61.0%であった．対象とする手法集合や検体集合，判定方法が異なるため直接比較はできないが，環境検査処理を広く候補として捉える基準では，本研究でも多数の検体が該当した．

Kim らの EvDetector は，環境検査結果が後続の条件分岐に影響することまでを確認し，2017 年から 2020 年の検体における割合を平均 16.21%と報告している [8]．本研究でこれに対応する Level 2 以上の割合は，2020–2021 年で 40.0%，2025–2026 年で 32.9%であった．この差には，対象とする環境検査処理や検体集合に加え，解析可能な実行経路の範囲など，解析方法の違いが影響している可能性がある．本研究の Level 3 では，さらに分岐後に悪性挙動が抑止されることまで確認しているため，Level 1 や Level 2 とは異なる範囲を判定対象としている．

### 5.2　年代間で観測された差異の解釈

本研究で比較した 2 時期では，VM・Sandbox 回避機能の確認割合に統計的に有意な差は確認されなかった．したがって，本研究の結果のみから，2020–2021 年から 2025–2026 年にかけて VM・Sandbox 回避機能の利用が増加ま

© 2026 Information Processing Society of Japan

**表 7** 時期別に確認された VM・Sandbox 回避手法と代表的な API・命令・指標

| 手法 | 2020–2021 | | 2025–2026 | |
|---|---|---|---|---|
| | 件数 | 代表 API・命令・指標 | 件数 | 代表 API・命令・指標 |
| Timing/emulation behavior | 24 | Sleep, GetTickCount, QueryPerformanceCounter, GetLastError | 4 | Sleep, GetTickCount, RDTSC, CPUID |
| VM/Sandbox artifacts | 7 | VirtualBox, VMware, SbieDll.dll, CPUID | 20 | VMware, VirtualBox, SbieDll.dll, Win32_ComputerSystem |
| Analysis tools/security products | 9 | taskmgr, procexp, bitdefender, TerminateProcess | 26 | TerminateProcess, OpenProcess, FindWindowA, taskmgr |
| System resources | 2 | CPUID, Win32_ComputerSystem | 12 | ManagementObjectSearcher, disk, DriveInfo.TotalSize |
| Host/user environment | 11 | keyboard layout, LANGID, username, InitSafeBootMode | 12 | GetComputerNameA, computer name, GetUserNameA |
| User activity | 1 | GetCursorPos, GetForegroundWindow, GetTickCount | 0 | 該当なし |
| API semantics | 4 | GetLastError, LoadCursorFromFileA | 1 | GetLastError, GetModuleHandleW |
| Runtime instrumentation | 0 | 該当なし | 3 | AMSI, ETW, Module32First, Module32Next |

たは減少したと結論づけることはできない．一方，各カテゴリの確認割合には異なる傾向が観測された．

2020–2021 年では Timing/emulation behavior が確認検体の 49.0%を占めたのに対し，2025–2026 年では 6.3%であった．このカテゴリには，Sleep や GetTickCount，QueryPerformanceCounter 等を用いて時間経過や実行時の挙動を確認し，解析環境を判定する処理が含まれる．この差の要因を本研究の結果のみから特定することはできないが，解析環境側における時間処理やエミュレーションへの対策，マルウェア側における環境判定方法の変化，検体集合の構成等が影響した可能性がある．過去の縦断調査でも，解析回避に利用される個々の手法の利用状況は時期によって変化することが報告されている [6]．したがって，本研究で観測された Timing/emulation behavior の差についても，回避機能全体の増減ではなく，解析環境を識別するために用いられる手掛かりの違いの一部として捉える必要がある．

一方，2025–2026 年では Analysis tools/security products が最も多く確認された．このカテゴリには，実行中のプロセスや Window，Service 等から AV，EDR，解析ツールを検出し，その結果に応じて悪性挙動を抑止する処理が含まれる．VM が実運用環境にも広く利用される中では，VM の存在のみを解析環境の指標とすると，本来攻撃対象となり得る環境まで除外する可能性がある．これに対し，解析ツールやセキュリティ製品の存在は，解析や監視が行われている環境をより直接的に示す手掛かりとなり得る．そのため，2025–2026 年の検体では，VM そのものの存在だけでなく，解析・監視用ソフトウェアを手掛かりとして実行環境を識別する実装が多く確認された可能性がある．

また，2025–2026 年では VM/Sandbox artifacts および System resources も比較的多く確認された．Maffia らは 2016 年から 2020 年にかけて VM Checks および Resource Profiling が減少する傾向を報告し，仮想環境が実運用環境として利用されるようになったことがその一因である可能性を考察している [7]．一方，本研究では，VMware や VirtualBox，SbieDll.dll 等の VM・Sandbox に由来する特徴や，ディスク容量等のシステム資源に関する特徴が 2025–2026 年でも確認された．両研究では対象とする手法集合や分類方法，検体集合が異なるため直接比較することはできないが，少なくとも本評価対象では，これらの特徴を利用した回避手法が 2025–2026 年においても確認された．特に，VMware や VirtualBox，SbieDll.dll 等の特徴は，仮想環境一般の存在ではなく，特定の VM や Sandbox 環境を識別する手掛かりとして利用されている可能性がある．

また，1 検体から複数カテゴリの手法が確認された例も存在した．このことから，解析環境の判定には，単一の環境情報だけでなく，仮想化環境の痕跡，解析・監視用ソフトウェア，システム資源，ホストやユーザの構成等，複数の環境情報が利用される場合があると考えられる．

以上の結果から，VM・Sandbox 回避機能の利用状況を把握する際には，回避機能の確認割合だけでなく，どのような環境上の特徴が解析環境の識別に利用されているかを継続的に把握することが重要である．また，解析環境の設計においても，VM 固有の痕跡だけでなく，解析ツールやセキュリティ製品，システム資源，ホスト・ユーザ環境等の多様な特徴について，解析環境を識別する手掛かりとなり得るかを考慮する必要がある．

## 6. 制限事項

本研究にはいくつかの制限事項がある．

第一に，本手法は Unprotect Project に基づいて構築した既知の VM・Sandbox 関連シグネチャを解析の起点としている．そのため，シグネチャに含まれない未知の環境検

© 2026 Information Processing Society of Japan

査処理や，既知の手法とは異なる実装を用いる回避機能を抽出できない可能性がある．したがって，本研究で得られた確認割合は，Windows マルウェア全体における VM・Sandbox 回避機能の真の保有率を示すものではない．

第二に，RQ1 では Level 2 以上に分類された検体を人手で解析し，Level 3 判定の適合率および再現率を評価したが，Level 0 および Level 1 に分類された検体については網羅的な人手解析を行っていない．そのため，シグネチャとの一致が得られなかった検体や，環境検査結果に依存する条件分岐を特定できなかった検体に VM・Sandbox 回避機能が含まれる可能性については評価できていない．本研究で示した再現率 100.0%は Level 2 以上の検体に対する Level 3 判定の再現率であり，解析対象検体全体に対する提案手法の再現率を示すものではない．

第三に，分岐後挙動の解析には LLM を用いており，その判定は LLM によるコードおよび制御フローの解釈に依存する．Level 3 と判定された 81 検体のうち 17 検体は，人手解析では VM・Sandbox 回避機能を確認できなかった．また，使用するモデルや解析時に取得可能な情報によって判定結果が変化する可能性がある．したがって，本研究で得られた判定性能は，使用したモデルおよび解析条件の下で得られた結果として解釈する必要がある．

第四に，本研究では両時期について同一条件で 10,000 検体の収集集合を作成し，そこから無作為に検体を抽出したが，解析数は 2020–2021 年で 400 検体，2025–2026 年で 659 検体と異なる．また，対象は VirusTotal への初回投稿日時および検知数の条件を満たす検体に限定される．VirusTotal への初回投稿日時は，検体の作成時期や実環境での出現時期を直接示すものではない．さらに，各期間の検体集合には同一ファミリに属する複数の検体が含まれ，両期間でファミリ構成も異なるが，本研究ではファミリ構成を統制した比較は行っていない．したがって，本研究で観測された確認割合や手法カテゴリの差異を Windows マルウェア全体へ一般化する際には注意が必要であり，特に手法カテゴリの差異は，本評価対象の 2 時期における差異として解釈する必要がある．

## 7. おわりに

本研究では，環境検査結果に依存する条件分岐とその分岐後の処理を解析し，解析環境を検出した経路で悪性挙動が抑止されるかに基づいて，Windows マルウェアの VM・Sandbox 回避機能を抽出する手法を提案した．

2025–2026 年の評価では，217 検体で環境検査結果に依存する条件分岐が確認され，分岐後挙動まで考慮することで 81 検体が Level 3 と判定された．人手解析ではこのうち 64 検体で VM・Sandbox 回避機能が確認され，Level 3 判定の適合率は 79.0%であった．このことから，本評価対象では，分岐後挙動を考慮することが回避を目的としない処理を候補から除外するのに有効であった．

また，2020–2021 年と 2025–2026 年の VM・Sandbox 回避機能の確認割合はそれぞれ 12.25%と 9.71%で，有意差は確認されなかった．一方，2020–2021 年では Timing/emulation behavior，2025–2026 年では Analysis tools/security products および VM/Sandbox artifacts が多く，手法構成には異なる傾向が観測された．

今後は，Level 0 および Level 1 も人手解析し，解析パイプライン全体の再現率を評価する．また，対象時期と解析検体数を拡大し，観測された差異が検体集合に依存するものか，長期的変化を反映するものかを検証する．

### 参考文献


[1] Lindorfer, M., Kolbitsch, C. and Milani Comparetti, P.: Detecting Environment-Sensitive Malware, *Recent Advances in Intrusion Detection*, Lecture Notes in Computer Science, Vol. 6961, Springer, pp. 338–357 (online), DOI: 10.1007/978-3-642-23644-0_18 (2011).

[2] Chen, X., Andersen, J., Mao, Z. M., Bailey, M. and Nazario, J.: Towards an Understanding of Anti-Virtualization and Anti-Debugging Behavior in Modern Malware, *Proceedings of the 38th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2008)*, IEEE, pp. 177–186 (online), DOI: 10.1109/DSN.2008.4630086 (2008).

[3] Kirat, D., Vigna, G. and Kruegel, C.: BareCloud: Bare-Metal Analysis-Based Evasive Malware Detection, *23rd USENIX Security Symposium (USENIX Security 14)*, USENIX Association, pp. 287–301 (2014).

[4] Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I. and Zaharia, M.: A View of Cloud Computing, *Communications of the ACM*, Vol. 53, No. 4, pp. 50–58 (online), DOI: 10.1145/1721654.1721672 (2010).

[5] Chandramouli, R.: Security Recommendations for Server-based Hypervisor Platforms, NIST Special Publication 800-125A Revision 1, National Institute of Standards and Technology (2018).

[6] Galloro, N., Polino, M., Carminati, M., Continella, A. and Zanero, S.: A Systematical and Longitudinal Study of Evasive Behaviors in Windows Malware, *Computers & Security*, Vol. 113, p. 102550 (online), DOI: 10.1016/j.cose.2021.102550 (2022).

[7] Maffia, L., Nisi, D., Kotzias, P., Lagorio, G., Aonzo, S. and Balzarotti, D.: Longitudinal Study of the Prevalence of Malware Evasive Techniques, *CoRR*, Vol. abs/2112.11289 (online), DOI: 10.48550/arXiv.2112.11289 (2021).

[8] Kim, M., Cho, H. and Yi, J. H.: Large-Scale Analysis on Anti-Analysis Techniques in Real-World Malware, *IEEE Access*, Vol. 10, pp. 75802–75815 (online), DOI: 10.1109/ACCESS.2022.3190978 (2022).

[9] 篠宮一真，岩村　誠：LLM エージェントを用いたバイナリ解析による IoT マルウェアの通信挙動抽出，情報処理学会研究報告， Vol. 2026-SPT-64, No. 85, pp. 1–8 (2026).

[10] Unprotect Project: Unprotect, `https://unprotect.it/` (2026). Version 7.0.7, snapshot: 2026-04-01, accessed: 2026-08-14.

[11] VirusTotal: VirusTotal Intelligence, `https://www.virustotal.com/` (2026). Accessed: 2026-08-14.


© 2026 Information Processing Society of Japan

# 付　　録

## A.1　使用プロンプト

本研究の自動解析において，VM・Sandbox 回避機能の判定に用いたプロンプトを以下に示す．実際の解析では，付録 A.3 に示す VM・Sandbox 関連 API および命令のシグネチャ一覧を併せて与えた．

```
You are an expert in Windows malware analysis. Using the
     Ghidra
analysis tools provided through MCP, analyze the target
     specimen
for VM/Sandbox evasion functionality.

# Goal

For each VM/Sandbox-related candidate, determine:

1. whether a candidate feature is present,
2. whether the result of an environment check controls a
   conditional branch, and
3. whether malicious behavior is suppressed on the path
   corresponding to an analysis environment.

The presence of an API, string, import, or instruction
     pattern
alone is not sufficient to confirm VM/Sandbox evasion.

Anti-Debugging is outside the scope of this analysis.

# Candidate Features

Candidate features are derived from Sandbox Evasion
     techniques
documented by the Unprotect Project.

Use the supplied API and instruction features only to
     identify
Level 1 candidates. Related strings may be used as
     supplementary
evidence, but a string alone is not sufficient to identify a
Level 1 candidate.

Do not treat a feature match alone as evidence that an
     evasion
technique is implemented.

[Candidate feature list omitted here; see Appendix A.3.]

# Analysis Levels

Level 0:
No candidate feature is identified.

Level 1:
A VM/Sandbox-related candidate feature is identified.

Level 2:
Confirm that the candidate is actually used as an environment
check and that its result, output value, or a derived value
controls a conditional branch.

The required evidence relationship is:

environment check
-> result or derived value
-> comparison / decision
-> conditional branch

Do not assign Level 2 merely because an environment-related
operation and a conditional branch occur near each other.
Confirm the data dependency between them.

Level 3:
For each Level 2 branch, first determine which branch
     corresponds
to the detected analysis environment and which corresponds to
      the
normal environment.

Then trace both the analysis-environment-detected path and
     the
normal path.

Continue tracing until the paths reconverge, execution
     terminates,
or their behavioral difference can be determined.

Assign Level 3 only when malicious behavior is suppressed on
     the
analysis-environment-detected path.

Suppression includes:

* early termination before malicious behavior is executed,
* skipping payload unpacking, decryption, or execution, or
* introducing a long delay that prevents malicious behavior
     from
  being observed within the analysis period.

Do not assign Level 3 for normal error handling,
     compatibility
checks, CPU feature selection, ordinary polling or waiting,
normal resource checks, or branches that later reconverge and
perform the same malicious behavior.

# Analysis Policy

Use disassembly, decompilation, cross-references, call graphs
     ,
and data-flow information to establish the evidence.

For candidate APIs, confirm their actual use. If a direct
     call
cannot be found, inspect wrappers, GetProcAddress resolution,
function pointers, resolver tables, and API hashing/export
resolution where necessary. If an API resolved through these
mechanisms matches a supplied candidate API, treat it as a
Level 1 candidate.

For instruction-based candidates, identify the concrete
instruction and trace the values produced by it.

For Level 2, provide the instruction sequence that connects
     the
environment-check result to the conditional branch.

For Level 3, provide evidence from:

1. the environment-dependent branch,
2. the analysis-environment-detected path, and
```

© 2026 Information Processing Society of Japan

```
3. the normal path.

Explain the semantic difference between the two paths and
     identify
which malicious behavior, if any, is suppressed.

Consider alternative non-evasion explanations for candidate
features. Do not raise the Level when normal program behavior
      is
better supported by the code.

Do not invent addresses or missing control flow. If the
     evidence
required for a higher Level cannot be established, retain the
lower Level and report the unresolved point.

The specimen Level is the maximum Level reached by any
     candidate.

# Analysis Status

determined:
The relevant candidates and post-branch paths were
     sufficiently
analyzed to determine whether Level 3 behavior is present.

inconclusive:
The presence or absence of Level 3 behavior cannot be
     determined
because important evidence remains unresolved, such as
     packing,
unresolved calls, unknown branch targets, unavailable payload
      code,
or incomplete post-branch paths.

Do not use determined when unresolved evidence could affect
     the
Level 3 decision.

# Verdict Rules

confirmed:
The analysis status is determined and at least one candidate
reaches Level 3.

not_confirmed:
The analysis status is determined and no candidate reaches
Level 3.

unknown:
The analysis status is inconclusive.

# Output Schema

Output valid JSON only.

{
"sample_level": 0,
"analysis_status": "determined | inconclusive",
"vm_sandbox_evasion":
"confirmed | not_confirmed | unknown",

"candidates": [
{
"feature": "<API / instruction>",
"level": 1,
  "environment_check":
    "<what environmental property is checked>",
...
  "branch_evidence": {
    "address": "<address | unknown>",
    "instructions":
      "<check result -> comparison -> conditional branch>",
    "interpretation":
      "<how the check result controls the branch>"
  },

  "detected_path_evidence": {
    "instructions":
      "<major instructions or calls on detected path>",
    "interpretation":
      "<behavior on detected path>"
  },

  "normal_path_evidence": {
    "instructions":
      "<major instructions or calls on normal path>",
    "interpretation":
      "<behavior on normal path>"
  },

  "suppressed_behavior":
    "<suppressed malicious behavior | none | unknown>",

  "unresolved":
    "<unresolved analysis point | none>"
}
...
],

"summary":
"<reason for the specimen-level classification>"
}
```

© 2026 Information Processing Society of Japan

## A.2　解析対象検体のファミリ分布

評価対象検体の構成を補足するため，各期間について収集した 10,000 検体の集合と，評価に用いた判定可能検体におけるマルウェアファミリの分布を示す．マルウェアファミリには，VirusTotal がアンチウイルス製品の検知名等から抽出した popular threat name を用いた．複数の名称が得られる場合には，最も多くの検知結果で確認された名称をファミリ名として用いた．

まず，収集検体集合における出現割合に応じてファミリを区分し，各区分に属するファミリのうち，評価対象検体に含まれたファミリの割合を図 A·1 および図 A·2 に示す．図中の Families は，各区分において評価対象検体に含まれたファミリ数と収集検体集合に存在するファミリ数を示す．Population share は，各区分に属するファミリが収集検体集合全体に占める検体割合を示す．Family coverage は，各区分に属するファミリのうち，評価対象検体に少なくとも 1 検体含まれたファミリの割合を示す．

両期間とも，収集検体集合における出現割合が高いファミリほど評価対象検体に含まれる割合が高く，出現割合が 0.50%以上のファミリは全て評価対象として含まれている．低頻度のファミリには評価対象検体に含まれていないものも多いが，これらの Population share は比較的小さく，評価対象検体には収集検体集合における主要なファミリが広く含まれていることがわかる．

また，各期間の収集検体集合における出現割合の上位 20 ファミリについて，収集検体集合と評価対象検体における各ファミリの割合を図 A·3 および図 A·4 に示す．2020–2021 年については 10,000 検体と判定可能な 400 検体，2025–2026 年については 10,000 検体と判定可能な 659 検体を比較する．

上位 20 ファミリについても，収集検体集合と評価対象検体の間で一部のファミリに割合の差は見られるものの，多くの主要ファミリでは構成割合は概ね近い値を示している．以上から，評価対象検体は低頻度のファミリを網羅するものではないが，各期間の収集検体集合における主要なファミリ構成については概ね反映されていることを確認できる．

## A.3　Level 1 候補抽出用シグネチャ

Level 1 の候補抽出に用いたシグネチャを表 A·1 に示す．シグネチャは，Unprotect Project に登録されている解析回避手法のうち，本研究の対象に該当する手法から抽出した API および命令で構成される．表中には，解析時に照合する API・命令と，そのシグネチャに対応する Unprotect Project 上の手法を示す．一つの API・命令が複数の手法で利用される場合には，対応する手法を併記した．これらのシグネチャは VM・Sandbox 回避機能の存在を直接判定

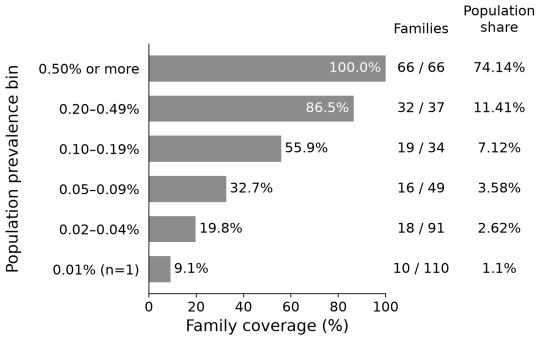



**図 A·1**　2020–2021 年の収集検体集合におけるファミリ出現割合別のカバレッジ

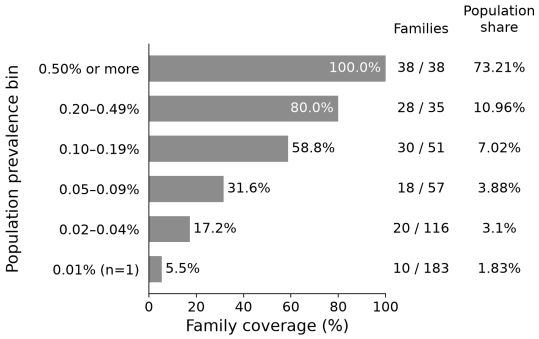



**図 A·2**　2025–2026 年の収集検体集合におけるファミリ出現割合別のカバレッジ

するものではなく，後続解析の起点となる Level 1 の候補抽出に用いる．

© 2026 Information Processing Society of Japan

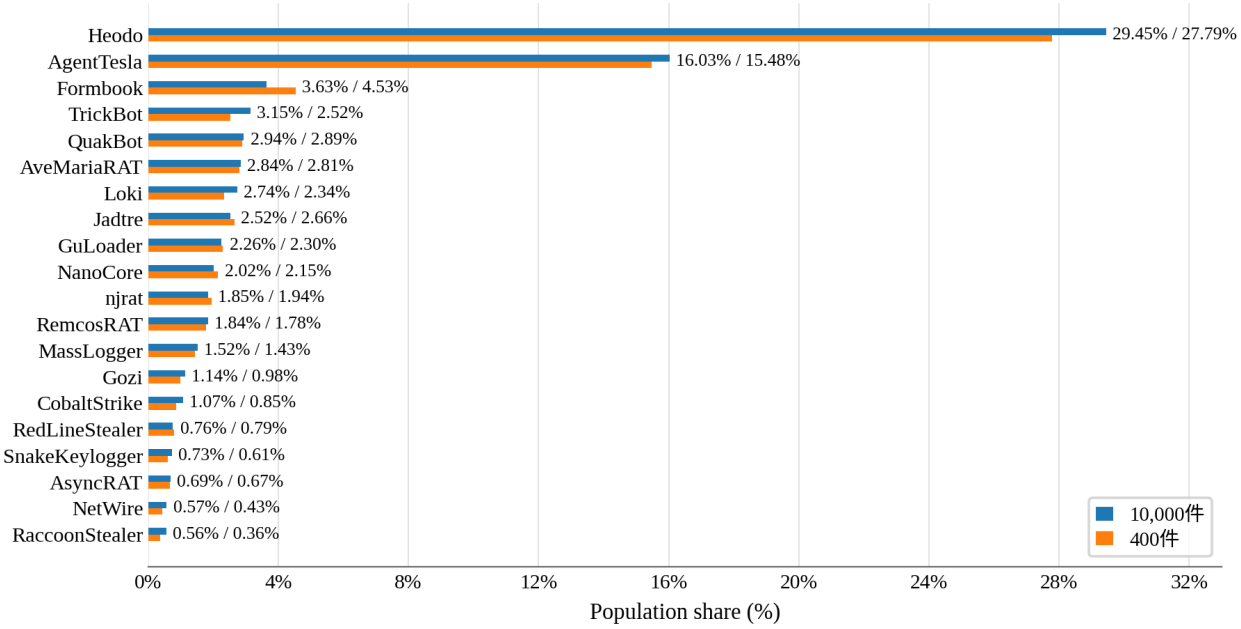



**図 A·3** 2020–2021 年の収集検体集合および評価対象検体における上位 20 ファミリの分布

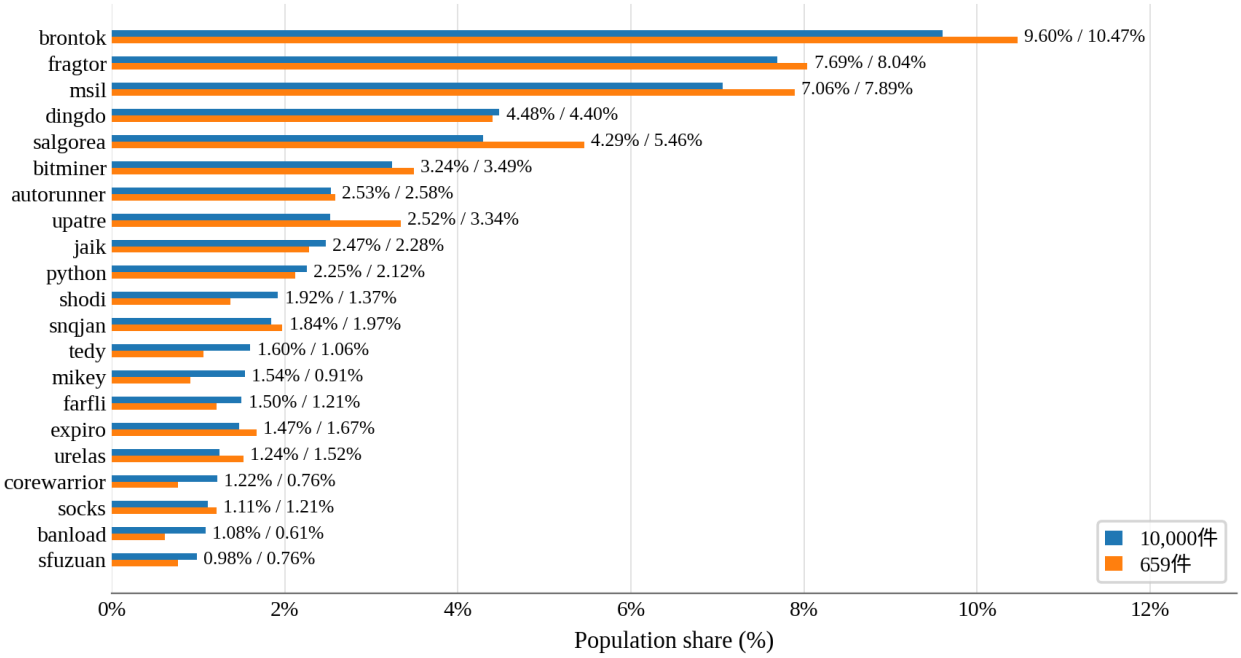



**図 A·4** 2025–2026 年の収集検体集合および評価対象検体における上位 20 ファミリの分布

© 2026 Information Processing Society of Japan

**表 A·1** Level 1 候補抽出に用いる Unprotect Project 由来のシグネチャ

| 種別 | 照合対象 | 対応する Unprotect 手法 |
|---|---|---|
| API | BuildCommDCBAndTimeoutsA | buildcommdcbandtimeoutsa |
| API | CreateNamedPipeA/W | checking-pipe |
| API | CreateToolhelp32Snapshot | detecting-running-process-enumprocess-api, detecting-virtual-environment-process |
| API | EnumPrintersA/W | connected-printer |
| API | EnumProcesses | detecting-running-process-enumprocess-api, detecting-virtual-environment-process |
| API | EnumServicesStatusA/W, EnumServicesStatusExA/W | detecting-active-services |
| API | EnumWindows, FindWindowA/W | detecting-window-with-findwindow-api |
| API | GetAdapterAddresses, GetAdaptersInfo | detecting-mac-address |
| API | GetComputerNameA/W | detecting-hostname-username |
| API | GetCursorPos | checking-mouse-activity, user-interaction-are-you-human |
| API | GetDesktopWindow | checking-screen-resolution |
| API | GetDiskFreeSpaceA/W, GetDiskFreeSpaceExA/W | checking-hard-drive-size |
| API | GetDriveTypeA/W | detecting-usb-drive |
| API | GetForegroundWindow | getforegroundwindow, user-interaction-are-you-human |
| API | GetLastInputInfo | checking-mouse-activity, user-interaction-are-you-human |
| API | GetLocalTime, GetSystemTime, NtQueryPerformanceCounter, QueryPerformanceCounter, QueryPerformanceFrequency, timeGetTime | getlocaltime-getsystemtime-timegettime-ntqueryperformancecounter |
| API | GetModuleFileNameA/W | checking-malware-name |
| API | GetNativeSystemInfo, GetSystemInfo | cpu-counting |
| API | GetSystemFirmwareTable | hyper-v-signature |
| API | GetSystemMetrics | checking-screen-resolution |
| API | GetTickCount, GetTickCount64 | gettickcount |
| API | GetUserNameA/W | detecting-hostname-username |
| API | GlobalMemoryStatusEx | checking-memory-size |
| API | LocalSize | localsize0 |
| API | MsgWaitForMultipleObjects | user-interaction-are-you-human |
| API | NetShareEnum, WNetEnumResourceA/W | vboxenumshares |
| API | NtDelayExecution | ntdelayexecution, onset-delay |
| API | NtQueryInformationProcess | ntqueryinformationprocess |
| API | NtQueryObject | ntqueryobject |
| API | NtQuerySystemInformation | odd-thread-count |
| API | NtSetInformationThread | ntsetinformationthread |
| API | Process32First/FirstW, Process32Next/NextW | detecting-running-process-enumprocess-api, detecting-virtual-environment-process |
| API | Sleep | onset-delay, stalling-code |
| API | SuspendThread | suspendthread |
| API | UnhandledExceptionFilter | unhandled-exception-filter |
| 命令 | `CPUID (0F A2)` | cpuid, vmcpuid, hyper-v-signature, bochs-cpu-oversights-evasion, qemu-cpu-brand-evasion |
| 命令 | `RDTSC (0F 31)` | rdtsc |
| 命令 | `RDTSCP (0F 01 F9)` | rdtscp |
| 命令 | SIDT, SMSW（`0F 01` 系） | sidt-red-pill, smsw |
| 命令 | SLDT, STR（`0F 00` 系） | sldt-no-pill, str |
| 命令 | `MOV EAX, 0x564D5868; ...; IN EAX, DX` | querying-the-io-communication-port |
| 命令 | `MOV DX, 0x5658; ...; IN EAX, DX` | querying-the-io-communication-port |

© 2026 Information Processing Society of Japan